

A PROJECT REPORT

ON

APIxIndia

A REAL-TIME WEB-BASED MESSAGING APPLICATION

Submitted by

**Gulafsha Khatoon
Mohammed Mubeen Khan**

Roll No: 26/CS-10 | Division: A

in partial fulfillment for the award of the degree of

BACHELOR OF SCIENCE

in

COMPUTER SCIENCE

Under the Guidance of

[Insert Guide Name Here]

Department of Computer Science



Bhartiya Gramin Punarrachana Sanstha's

MUMBAI COLLEGE OF ARTS, COMMERCE & SCIENCE

J.K. Jadhav Knowledge Centre, Nadkarni Park, Wadala (E), Mumbai - 400 037

Academic Year: 2026–2027

CERTIFICATE

This is to certify that Gulafsha Khatoon Mohammed Mubeen Khan(Roll No: 26/CS-10, Division: A), a student of T.Y.B.Sc. Computer Science, has satisfactorily completed the project entitled “APIxIndia A REAL-TIME WEB-BASED MESSAGING APPLICATION ” during the academic year 2026–2027, in partial fulfillment of the requirements for the award of the Bachelor of Science degree in Computer Science.

The work presented in this report is submitted for academic evaluation and demonstrates the design and implementation of a real-time messaging application using Python, Playwright/Scrapy, Redis/Celery, PostgreSQL with TimescaleDB, FastAPI and a React-based dashboard.

Project Guide

Signature of Examiner

Head / In-charge

Principal

DECLARATION

I, Gulafsha Khatoon Mohammed Mubeen Khan, hereby declare that the project entitled “APIxIndia A REAL-TIME WEB-BASED MESSAGING APPLICATION” submitted in partial fulfillment for the award of Bachelor of Science in Computer Science is my academic project work. The report describes the implementation and design of the application and has been prepared for academic submission.

I further declare that the information presented in this report is intended to describe the implemented project and its current capabilities. External technologies, frameworks and documentation used in the development have been acknowledged in the References section.

Student Signature: _____

Name: Gulafsha Khatoon Mohammed Mubeen Khan

Roll No.: 26/CS-10

Place: Mumbai

Date: 22 September 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, faculty members and the Department of Computer Science for their guidance and support during the development of this project.

I am thankful to my college for providing the academic environment and resources required to study web development, databases, networking and software engineering concepts. I also thank my classmates, friends and family for their encouragement and for helping me test the application during development.

Finally, I would like to acknowledge the open-source technologies and documentation that helped me understand and implement the project.

Gulafsha Khatoon Mohammed Mubeen Khan

T.Y.B.Sc. Computer Science

ABSTRACT

APIx-India is a proposed real-time airfare price indexing platform designed to collect, clean, structure and analyse publicly visible domestic flight prices at a much higher frequency than a traditional periodic survey. The central idea is to treat online airfare as high-frequency economic data and transform it into a reproducible price index that can support inflation analysis, transport monitoring and policy research.

The system is designed to collect prices from airline websites and online travel agencies across important domestic routes and multiple booking windows. The proposed sampling model uses five advance-purchase horizons - $T+1$, $T+7$, $T+15$, $T+30$ and $T+45$ - so that last-minute and advance-booking behaviour can be observed together. The supplied concept targets more than 150,000 price observations per day across a multi-source network.

The processing pipeline separates fare components where the source exposes them, performs validation and anomaly detection, handles unavailable observations explicitly, and computes a daily Airfare Price Index (APIx). The analytical layer is intended to support carrier, route, booking-window and time-based comparisons while preserving an auditable record of the source observation and collection time.

A dashboard is proposed for government and research users, with heatmaps, trend lines, surge alerts, route comparisons and downloadable data. A secure API layer is designed to expose standardised series for further statistical processing. The architecture uses Python, Playwright/Scrapy, Redis/Celery, PostgreSQL with TimescaleDB, FastAPI and a React-based dashboard.

The concept is intended as a college-scale prototype and research demonstration. Claims about production-scale coverage, legal compliance, forecast accuracy and operational savings should be validated through controlled testing and formal institutional review before real-world deployment.

Keywords

APIx-India, airfare price index, CPI augmentation, web data collection, real-time pricing, Python, FastAPI, Playwright, PostgreSQL, TimescaleDB, machine learning, SDMX, data quality.

TABLE OF CONTENTS

1. Introduction	7
2. Existing and Proposed System	8
3. Requirements and Objectives	9
4. Technology Stack and System Architecture	10
5. Data Acquisition and Database Design	11
6. Data Processing and Index Methodology	12
7. Dashboard and Government Integration	13
8. Major Functionalities	14
9. Testing and Results	15
10. Security, Ethics and Legal Considerations	16
11. Limitations	17
12. Future Scope	18
13. Conclusion	19
References	20
Glossary	21
Appendix - API and Data Pipeline Summary	22

CHAPTER 1: INTRODUCTION

1.1 Background

Airline ticket prices are dynamic. The same route may show different prices depending on the date of travel, time of booking, remaining inventory, demand conditions, fare rules and the channel through which the booking is made. This behaviour makes airfare a useful example of a market where a single monthly observation may not describe the full movement of prices experienced by consumers.

APIx-India proposes an automated approach in which price observations are collected repeatedly from online sources and converted into a structured time series. Instead of treating a ticket as one static price, the system records the price together with route, carrier, travel date, booking horizon, source, collection timestamp and other available fare attributes.

The project therefore combines web data collection, database engineering, data quality methods, statistical index construction and information visualisation in one integrated application. The design is deliberately modular so that data collection can be changed without rewriting the index engine or dashboard.

1.2 Problem Statement

The problem addressed by the project is the difficulty of representing fast-changing airfare prices with low-frequency observations. Manual or periodic data collection can miss short-lived surges, discount periods, route-specific variation and differences between advance-purchase windows. A second problem is that raw web observations are noisy: sources may expose duplicate offers, temporary glitches, unavailable flights or differently structured fare components.

APIx-India addresses these issues by creating a repeatable collection pipeline, a common data schema and an index methodology that can operate on the resulting high-frequency dataset.

1.3 Objectives

- To design an automated pipeline for collecting domestic airfare observations from multiple online sources.
- To cover major routes and multiple advance-purchase windows rather than using one booking date only.
- To standardise and validate fare records while preserving source and timestamp information.
- To calculate a daily airfare price index with transparent weighting rules.
- To provide a dashboard for route trends, price surges and data-quality monitoring.
- To expose structured data through a secure API for downstream statistical analysis.
- To document security, ethical collection, limitations and reproducibility requirements.

1.4 Scope

The project focuses on domestic scheduled air travel and is intended for academic demonstration, prototype evaluation and research exploration. It can later be extended to other transport and hospitality prices, but such extensions are outside the current implementation scope.

CHAPTER 2: EXISTING AND PROPOSED SYSTEM

2.1 Existing Approach

A low-frequency price survey typically collects a limited set of observations at scheduled intervals and uses the observations available at that point in time. For an airfare market, this can be difficult because prices may move several times within a day and because consumers book the same route at different lead times.

An online search by a human observer also introduces operational limitations. It is difficult to repeat the same query at scale, difficult to preserve the exact collection timestamp, and difficult to compare hundreds of routes and booking windows consistently.

2.2 Proposed System

APIx-India replaces the repeated manual query process with an automated collection and processing pipeline. The collector retrieves public fare observations from configured airline and travel sources, maps the source data into a common record structure, validates the observations and stores them in a time-series capable database. The index engine then aggregates valid observations into route, carrier and booking-window series.

Area	APIx-India Approach
Collection	Scheduled automated collection with source and timestamp metadata
Sources	Airline websites and online travel agencies
Coverage	Major domestic routes across multiple advance-purchase windows
Processing	Validation, deduplication, anomaly detection and fare-component handling
Database	PostgreSQL with TimescaleDB-style time-series organisation
Analytics	Daily APIx index with transparent route and booking-window weights
Delivery	Dashboard plus secure API for structured data access
Governance	Audit trail, source tagging, retention and data-quality monitoring

2.3 Advantages

- Higher-frequency observations can reveal short-term price changes.
- Multiple sources reduce dependence on a single website.
- Booking-window stratification better reflects different consumer planning horizons.
- Persistent storage creates an auditable historical dataset.
- Dashboard and API outputs make the data usable by analysts and decision-support systems.

CHAPTER 3: REQUIREMENTS AND OBJECTIVES

3.1 Functional Requirements

- The system shall maintain a configurable list of routes, carriers, travel dates and booking windows.
- The collector shall schedule data retrieval jobs and record collection timestamps.
- Each observation shall retain source, route, carrier, travel date, booking horizon and fare fields when available.
- The processing service shall identify duplicates and observations outside configured validation rules.
- The index engine shall generate daily route and national-level series from approved observations.
- The dashboard shall show trends, route comparisons, heatmaps, source coverage and alerts.
- Authorised users shall be able to export aggregated data and retrieve approved series through an API.
- The system shall retain an audit record of processing status and data-quality events.

3.2 Non-Functional Requirements

- Reliability: scheduled jobs should retry transient failures without duplicating records.
- Scalability: the data model should support millions of time-stamped observations.
- Maintainability: collectors, processing, analytics and presentation should be loosely coupled.
- Reproducibility: index calculations should be parameterised and versioned.
- Security: credentials, API keys and administrative functions should be protected.
- Ethics: collection should respect applicable terms, access controls, rate limits and institutional review requirements.

3.3 Hardware and Software Requirements

Requirement	Prototype Specification
<i>Processor</i>	<i>Modern multi-core processor</i>
<i>Memory</i>	<i>8 GB RAM recommended for local development</i>
<i>Storage</i>	<i>20 GB or more for development data; scalable storage for long-term history</i>
<i>Operating System</i>	<i>Windows, Linux or macOS for development; Linux preferred for server deployment</i>
<i>Python</i>	<i>Python 3.10+</i>
<i>Browser Automation</i>	<i>Playwright / compatible headless browser</i>
<i>Database</i>	<i>PostgreSQL; TimescaleDB extension where appropriate</i>
<i>Dashboard</i>	<i>React.js and Tailwind CSS</i>

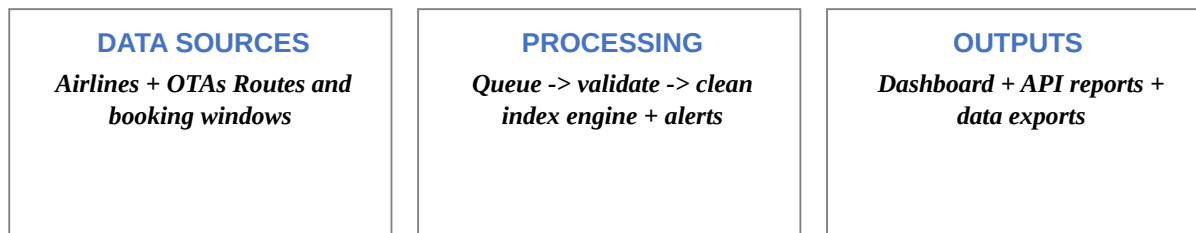
CHAPTER 4: TECHNOLOGY STACK AND SYSTEM ARCHITECTURE

4.1 Technology Stack

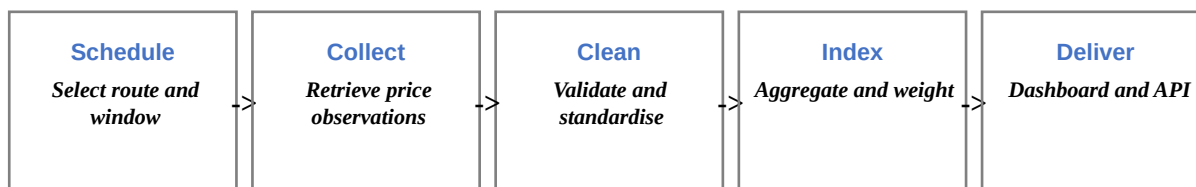
Technology	Purpose
<i>Python</i>	<i>Core programming language for collectors, processing and services</i>
<i>Playwright / Scrapy</i>	<i>Browser automation and structured web data collection</i>
<i>Redis + Celery</i>	<i>Job queue, scheduling and controlled parallel execution</i>
<i>Pandas + scikit-learn</i>	<i>Tabular processing and anomaly-detection workflows</i>
<i>PostgreSQL + TimescaleDB</i>	<i>Persistent relational and time-series storage</i>
<i>FastAPI</i>	<i>Secure data and analytics API layer</i>
<i>React.js + Tailwind CSS</i>	<i>Interactive dashboard and administrative interface</i>
<i>Charts / Map components</i>	<i>Price trends, route heatmaps and alert visualisation</i>

4.2 High-Level Architecture

The proposed architecture separates data acquisition from analytics and presentation. Collection workers create raw observations, a processing layer standardises and validates the records, the database stores both observation-level and aggregated data, and the API serves controlled access to the dashboard and authorised consumers.



4.3 Collection and Processing Flow



4.4 Design Principle

Every transformed record should remain traceable to a source observation. This makes it possible to distinguish raw collection failures from later analytical decisions and to reproduce an index value using the same configuration and data snapshot.

CHAPTER 5: DATA ACQUISITION AND DATABASE DESIGN

5.1 Data Acquisition Model

The collection model uses a matrix of route x source x carrier x travel date x booking window. The supplied project concept proposes five booking windows: T+1, T+7, T+15, T+30 and T+45. Each scheduled job records the query configuration and collection timestamp so that later analysis can separate market movement from collection timing.

Field	Description
<i>source_id</i>	<i>Configured airline or travel portal source</i>
<i>carrier</i>	<i>Operating airline shown for the offer</i>
<i>origin</i>	<i>Departure airport or city code</i>
<i>destination</i>	<i>Arrival airport or city code</i>
<i>travel_date</i>	<i>Date for which the fare is observed</i>
<i>booking_window</i>	<i>Days between collection date and travel date</i>
<i>collection_ts</i>	<i>Exact timestamp of the observation</i>
<i>base_fare</i>	<i>Base fare when exposed by the source</i>
<i>taxes_fees</i>	<i>Taxes and airport/other fees when available</i>
<i>total_price</i>	<i>Consumer-facing total fare used for index processing</i>
<i>availability_status</i>	<i>Available, sold-out, unavailable or uncertain</i>

5.2 Database Design

A relational schema can separate configuration, raw observations, cleaned observations, index outputs and audit information. A time-series extension is useful for efficient queries over collection timestamps and daily index values.

Table / Logical Entity	Purpose
<i>sources</i>	<i>Source configuration, access metadata and status</i>
<i>routes</i>	<i>Origin-destination master data and route weights</i>
<i>collection_jobs</i>	<i>Scheduled job definitions and execution status</i>
<i>raw_prices</i>	<i>Source-level raw observations before transformation</i>
<i>clean_prices</i>	<i>Validated and standardised observations</i>
<i>index_values</i>	<i>Daily route, carrier and aggregate APIx series</i>
<i>quality_events</i>	<i>Anomalies, rejected records and processing notes</i>
<i>api_users</i>	<i>Authorised application identities and access policy</i>

5.3 Data Retention

Raw data should be retained for a defined research and audit period, subject to storage cost and applicable source restrictions. Cleaned and aggregated series can be retained longer because they support longitudinal analysis and reproducibility.

CHAPTER 6: DATA PROCESSING AND INDEX METHODOLOGY

6.1 Data Cleaning Pipeline

- Schema validation: confirm required fields and expected data types.
- Deduplication: identify identical offers or repeated captures from the same job.
- Range validation: flag impossible, missing or extreme values for review.
- Component handling: preserve total price while separately storing base fare and fees when available.
- Availability handling: distinguish genuine sold-out conditions from technical collection failure.
- Anomaly detection: apply rule-based thresholds and, where justified, an Isolation Forest or similar model to identify unusual records.

6.2 Booking-Window Stratification

A central feature of APIx-India is the explicit treatment of booking horizon. T+1 reflects last-minute travel, while T+45 captures earlier purchase conditions. Keeping the windows separate prevents the index from being dominated by whichever booking horizon happens to contain the most observations and allows analysts to examine how price behaviour changes as departure approaches.

6.3 Index Construction

The project concept proposes a geometric-mean style aggregation. A generic formulation for a daily series is:

$$\text{Index}_t = 100 \times \exp\left(\sum(w_i \times \ln(P_{i,t} / P_{i,\text{base}}))\right)$$
 where $P_{i,t}$ is the cleaned observed price for item or route stratum i at time t , $P_{i,\text{base}}$ is its reference-period price, and w_i represents the chosen route, carrier and booking-window weights. In an implementation, the exact weighting structure, base period, chain-linking approach and treatment of new or missing items must be fixed in version-controlled methodology documentation.

6.4 Weighting Strategy

- Route weights can reflect passenger traffic or another documented measure of route importance.
- Booking-window weights can represent the intended consumer sampling design.
- Carrier weights can reflect route-level market representation rather than a single national average.
- Weights should be normalised and reviewed when route availability changes materially.

6.5 Quality Controls

The index engine should not silently replace missing data with a prediction. Imputed values, when used for analytical experiments, should be clearly labelled and excluded from official series unless a formal methodology approves the treatment. This distinction is important for statistical transparency.

CHAPTER 7: DASHBOARD AND GOVERNMENT INTEGRATION

7.1 Dashboard Overview

The dashboard is designed as a single web interface for monitoring collection health and interpreting price movements. The home view can present the current APIx value, daily change, route-level alerts, source coverage and data-quality indicators.

MONITOR <i>Jobs, source status, coverage and errors</i>	ANALYSE <i>Trends, heatmaps, route and window views</i>	EXPORT <i>CSV/JSON series, versioned API output</i>
---	---	---

7.2 Major Dashboard Components

- National trend: daily APIx movement with comparison against the selected base period.
- Route heatmap: visual comparison of price changes across major origin-destination pairs.
- Booking-window view: separate T+1, T+7, T+15, T+30 and T+45 series.
- Carrier panel: compare observed prices and source coverage without exposing sensitive operational credentials.
- Surge alerts: flag unusual changes using configurable thresholds and confidence rules.
- Data-quality panel: collection success rate, duplicate rate, missingness and anomaly counts.

7.3 API and Data Exchange

A FastAPI service can expose authenticated endpoints for approved aggregate series, metadata and quality measures. JSON is suitable for application integration, while CSV supports research workflows. The project concept also proposes SDMX-compatible exchange so that downstream statistical systems can map APIx series into a standardised data structure. The exact schema should be agreed with the receiving institution rather than assumed.

7.4 User Roles

Role	Typical Access
<i>Administrator</i>	<i>Source configuration, route list, job controls and access management</i>
<i>Analyst / Economist</i>	<i>Index series, trends, route analysis and exports</i>
<i>Data Quality Reviewer</i>	<i>Failed jobs, anomaly review and audit records</i>
<i>Read-only Viewer</i>	<i>Dashboard summaries and approved public indicators</i>

CHAPTER 8: MAJOR FUNCTIONALITIES

Functionality	Description	Status / Scope
<i>Source Configuration</i>	<i>Maintain airline/OTA source definitions, route lists and schedules.</i>	<i>Implemented / Prototype</i>
<i>Automated Collection</i>	<i>Run scheduled collection jobs for configured route-window combinations.</i>	<i>Implemented / Prototype</i>
<i>Raw Data Capture</i>	<i>Store observations with source and collection metadata.</i>	<i>Implemented / Prototype</i>
<i>Cleaning Pipeline</i>	<i>Validate, standardise and flag abnormal observations.</i>	<i>Implemented / Prototype</i>
<i>APIx Index Engine</i>	<i>Generate daily weighted series from approved observations.</i>	<i>Core feature</i>
<i>Route Analytics</i>	<i>Compare price movement across selected origin-destination pairs.</i>	<i>Core feature</i>
<i>Booking-Window Analytics</i>	<i>Compare T+1 through T+45 behaviour.</i>	<i>Core feature</i>
<i>Heatmap and Alerts</i>	<i>Highlight route-level changes and unusual movements.</i>	<i>Core feature</i>
<i>Government API</i>	<i>Expose controlled aggregate data through authenticated endpoints.</i>	<i>Prototype / Extension</i>
<i>Audit Trail</i>	<i>Track jobs, validation decisions and processing versions.</i>	<i>Core feature</i>

8.1 Important Implementation Boundary

The supplied idea describes a working prototype and reports a target collection volume above 150,000 prices every 24 hours. In an academic report, these figures should be presented together with reproducible run logs, source lists, time ranges and test scripts. The system should not claim complete national coverage merely because a multi-source sample has been collected.

The prototype should also distinguish between three layers of evidence: collected source observations, transformed analytical records, and derived index outputs. This makes it easier for an evaluator to trace an APIx value back to the inputs used to calculate it.

8.2 Error Recovery

- Retry temporary network or server failures with exponential backoff.
- Mark persistent source failure rather than inserting artificial observations.
- Keep job-level logs so that collection gaps are visible to analysts.
- Allow a route/source combination to be paused without stopping the complete pipeline.

CHAPTER 9: TESTING AND RESULTS

Testing should cover both technical correctness and statistical behaviour. The following test cases are suitable for prototype evaluation.

Test ID	Test Case	Expected Result
TC-01	<i>Run a scheduled collection for a valid route</i>	<i>Observations are saved with source and timestamp metadata.</i>
TC-02	<i>Repeat the same collection job</i>	<i>Duplicates are detected or safely versioned.</i>
TC-03	<i>Return an empty / sold-out source response</i>	<i>Status is recorded; no false price is inserted.</i>
TC-04	<i>Submit a clearly invalid price</i>	<i>Validation flags the record and excludes it from the index.</i>
TC-05	<i>Process a normal batch</i>	<i>Clean records are loaded into the analysis table.</i>
TC-06	<i>Calculate a daily route index</i>	<i>Output is reproducible from the same data snapshot and weights.</i>
TC-07	<i>Compare booking windows</i>	<i>T+1 to T+45 values remain separately identifiable.</i>
TC-08	<i>Open dashboard for a selected date range</i>	<i>Charts and heatmaps load without changing source data.</i>
TC-09	<i>Call API with valid credentials</i>	<i>Approved series are returned in documented format.</i>
TC-10	<i>Call protected endpoint without permission</i>	<i>Request is rejected and logged.</i>
TC-11	<i>Simulate source failure</i>	<i>Job enters a failed/retry state without corrupting stored history.</i>
TC-12	<i>Re-run a historical calculation</i>	<i>Same methodology version produces the same series.</i>

9.1 Prototype Measurements

The supplied project concept reports three headline prototype measurements: more than 150,000 price observations per 24-hour cycle, a 30-day comparison described as 95.2% correlation with reference tariff observations, and an observed lead time described as 21 days before a monthly report. These numbers are treated here as reported prototype results rather than independently verified statistical evidence. A final academic submission should include the raw evaluation dataset, calculation notebook or script, metric definitions, comparison baseline and test period so that the results can be reproduced.

9.2 Result Interpretation

A high collection volume does not automatically imply a representative index. Coverage, duplicate handling, missingness, route weights and source changes can materially affect the result. For this reason, the project should report both the index value and the quality indicators that explain how much data supported that value.

CHAPTER 10: SECURITY, ETHICS AND LEGAL CONSIDERATIONS

10.1 Security Measures

- Administrative credentials and API secrets should be stored outside source code.
- API authentication should use short-lived credentials or another documented access mechanism.
- Database access should be separated by role and limited to required operations.
- Transport should use HTTPS for deployed systems and secure connections for protected APIs.
- Logs should avoid storing unnecessary personal or credential information.
- Backups and database migrations should be tested before production use.

10.2 Ethical Data Collection

The project is designed around publicly visible fare information. Public visibility does not by itself remove all restrictions on automated collection. A deployment should therefore review each source terms of service, robots directives where applicable, technical access controls, rate limits and any institutional procurement or data-use requirements. The system should use the minimum collection frequency needed for the analytical objective and should avoid collecting passenger identities, payment details or other private information.

10.3 Legal and Governance Review

Because the project is intended to produce information that may be used in public statistics or policy analysis, deployment requires formal governance. The institution should document the lawful basis for collection, data retention periods, source agreements where required, incident response and ownership of derived datasets. Statements such as "100% legal" should not be made without a legal review of the specific deployment and sources.

10.4 Bias and Fairness

An airfare index can be biased by source availability, route selection, missing low-cost offers, sold-out flights and changes in airline distribution. The methodology should therefore publish coverage statistics and sensitivity checks showing how the index changes under alternate route or source selections.

10.5 Transparency

Every daily index release should carry a methodology version, coverage summary, data-quality status and calculation timestamp. This creates a clear distinction between a measured observation and an analytical estimate.

CHAPTER 11: LIMITATIONS

- Online fare data can change during a collection session, so a captured quote is a time-specific observation rather than a permanent price.
- Different websites may display different combinations of fare, taxes, fees and optional services, which complicates direct comparison.
- A source may change its page structure, network responses or access policy, requiring collector maintenance.
- Some flights may be unavailable at a booking horizon, creating non-random missing data.
- Sold-out observations should not automatically be replaced with modelled prices because imputation can change the statistical meaning of the index.
- Route selection and weights strongly influence the resulting series; a limited prototype sample is not automatically nationally representative.
- High-frequency collection creates storage, monitoring and engineering costs that grow with the number of sources and routes.
- Use of rotating proxies or automated browsing must be assessed against source restrictions and institutional policy rather than treated as a guaranteed solution to blocking.
- The proposed API output is a research and decision-support layer; it should not be described as an official CPI replacement unless adopted and governed by the relevant statistical authority.

11.1 Prototype vs Production

The current concept is strongest as a prototype architecture and methodology demonstration. A production national system would need service-level monitoring, source agreements where necessary, independent statistical review, stronger access controls, documented change management and long-term operational ownership.

11.2 Statistical Limitations

The geometric-mean formulation is only one possible index design. Results may differ if weights, base periods, linking rules, missing-value treatment or sampling methods change. These alternatives should be tested during validation rather than hidden behind a single headline number.

CHAPTER 12: FUTURE SCOPE

- Expand the route universe beyond the initial major-route sample and add regional connectivity measures.
- Add stronger source abstraction so new airlines and travel portals can be onboarded through configuration rather than custom code.
- Introduce a streaming architecture for selected high-priority routes while keeping scheduled bulk collection for the remaining sample.
- Add probabilistic missing-data models for research comparison, clearly separated from the published observed series.
- Develop automated methodology tests that compare index stability under alternate weights and route samples.
- Add multilingual and mobile-friendly dashboard views for wider institutional use.
- Provide versioned SDMX-compatible data structures and documented metadata mappings for statistical systems.
- Extend the same architecture to railway fares, intercity buses, hotel tariffs and other fast-changing consumer prices.
- Build alerting workflows for unusual route-level price changes and data-collection failures.
- Introduce independent audit reports and reproducible research packages for each published index release.

12.1 Long-Term Research Direction

A longer-term version of APIx-India could move from a single indicator toward a family of transport price measures: national airfare index, route-specific indices, advance-purchase indices and volatility indicators. Research can then examine whether different consumer groups experience different price paths depending on how far in advance they purchase tickets.

12.2 Scale-Out Architecture

For a multi-worker deployment, a distributed message broker can coordinate collectors and prevent duplicate work. A partitioned time-series database can manage larger history, while a data warehouse can support long-horizon research. These components should be introduced only when actual workload measurements justify the additional operational complexity.

CHAPTER 13: CONCLUSION

APIx-India presents a practical approach to a difficult measurement problem: airline fares are dynamic, source-dependent and strongly influenced by booking timing, yet a useful index requires consistent, repeatable and auditable observations. The proposed system addresses this by combining automated collection, standardised records, quality control, time-series storage, transparent weighting and visual analytics.

The project brings together important Computer Science concepts including web automation, asynchronous job processing, database design, API development, data cleaning, anomaly detection, statistical aggregation and user-interface engineering. Its architecture also demonstrates how raw online observations can be transformed into a controlled analytical product rather than being used as unstructured web data.

The most important design principle is traceability. A published index value should be explainable in terms of the routes, booking windows, sources, observations and methodology version that generated it. This is especially important when a system is intended to support public statistics or policy research.

The supplied concept reports promising prototype measurements, including a collection target above 150,000 observations per day and a 30-day reference comparison. These figures provide useful evaluation goals, but the academic strength of the project will depend on reproducible experiments, transparent methodology and careful treatment of missing and anomalous observations.

In conclusion, APIx-India can serve as a strong college-scale demonstration of high-frequency data engineering and applied price measurement. With further validation, governance and scaling, the same technical framework could support a wider family of digital price indicators across transportation and other dynamic consumer markets.

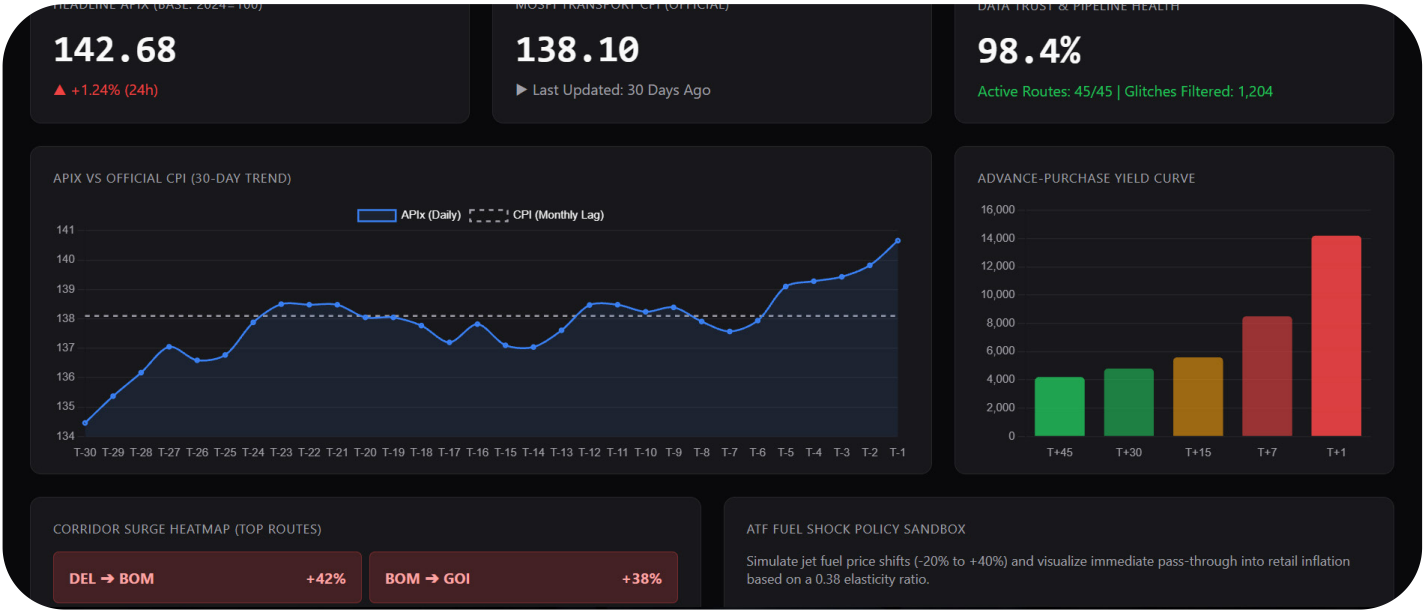
REFERENCES

1. Fette, I. and Melnikov, A., The WebSocket Protocol, RFC 6455, IETF, 2011.
2. FastAPI Documentation, official framework documentation and API reference.
3. Python Documentation, Python Standard Library and asynchronous programming documentation.
4. PostgreSQL Documentation, relational database and indexing documentation.
5. Timescale Documentation, time-series database concepts and hypertable documentation.
6. Playwright Documentation, browser automation and testing documentation.
7. Scrapy Documentation, web crawling and structured data extraction documentation.
8. Redis Documentation, in-memory data structures and queueing use cases.
9. Celery Documentation, distributed task queue and scheduled job processing documentation.
10. pandas Documentation, data structures and tabular data processing documentation.
11. scikit-learn Documentation, anomaly detection and Isolation Forest documentation.
12. MDN Web Docs, HTTP, Fetch API, browser storage and Web Platform documentation.
13. International Labour Office, IMF, OECD and World Bank, Consumer Price Index Manual: Concepts and Methods.
14. Reserve Bank of India, monetary policy and inflation measurement publications.
15. Ministry of Statistics and Programme Implementation, Consumer Price Index methodology and statistical publications.
16. Directorate General of Civil Aviation, domestic aviation traffic and tariff-related publications.
17. Cavallo, A. and Rigobon, R., “The Billion Prices Project: Using Online Prices for Measurement and Research,” Journal of Economic Perspectives, 30(2), 2016.
18. Statistical Data and Metadata eXchange (SDMX), standard information model and data exchange documentation.

GLOSSARY

Term	Meaning
<i>APIx</i>	<i>Airfare Price Index proposed by the project.</i>
<i>API</i>	<i>Application Programming Interface used for controlled software-to-software communication.</i>
<i>CPI</i>	<i>Consumer Price Index used to measure changes in consumer prices.</i>
<i>OTA</i>	<i>Online Travel Agency through which travel products can be searched or booked.</i>
<i>T+1 / T+7 / T+15 / T+30 / T+45</i>	<i>Booking-window labels representing the number of days before travel.</i>
<i>Web Scraping / Web Data Collection</i>	<i>Automated retrieval of information from web-accessible sources.</i>
<i>Playwright</i>	<i>Browser automation framework used to interact with web pages programmatically.</i>
<i>Celery</i>	<i>Distributed task-queue system for asynchronous background jobs.</i>
<i>Redis</i>	<i>In-memory data platform commonly used for queues, caching and coordination.</i>
<i>PostgreSQL</i>	<i>Open-source relational database management system.</i>
<i>TimescaleDB</i>	<i>Time-series capabilities built on PostgreSQL for time-oriented workloads.</i>
<i>Isolation Forest</i>	<i>Unsupervised anomaly-detection method used to identify unusual observations.</i>
<i>Jevons Index</i>	<i>Geometric-mean price index formulation.</i>
<i>SDMX</i>	<i>Standard for exchanging statistical data and metadata.</i>
<i>Nowcasting</i>	<i>Estimation of current conditions using data available before a conventional reporting cycle.</i>
<i>Audit Trail</i>	<i>Record of system actions, data changes and processing decisions used for traceability.</i>
<i>Sold-out Observation</i>	<i>A source response indicating that an offer is not currently available, which is not the same as a zero price.</i>
<i>Data Quality</i>	<i>Set of measures describing completeness, validity, consistency and reliability of observations.</i>

INTERFACE



Live Link

<https://exquisite-licorice-93de1d.netlify.app/>

APPENDIX - API AND DATA PIPELINE SUMMARY

A.1 Suggested API Categories

Category	Examples / Purpose
Sources	List active sources, source health and supported fields.
Routes	Create, update and query monitored origin-destination pairs.
Collection Jobs	Start, stop, retry and inspect scheduled collection runs.
Prices	Retrieve approved aggregated or research-level price observations.
Index	Get daily APIx values by date, geography, route, carrier or booking window.
Quality	Return coverage, missingness, duplicate and anomaly statistics.
Alerts	Read configured surge or collection-health alerts.
Metadata	Return methodology version, base period, weights and field definitions.

A.2 Example Processing Sequence

- Select configured routes and booking windows for the scheduled time slot.
- Create a collection job and queue source-specific tasks.
- Collect observations and store the raw response with timestamp and source metadata.
- Standardise fields and derive common route, carrier and fare representations.
- Run validation, deduplication and anomaly checks.
- Publish approved records to the clean-price table.
- Calculate daily index components and aggregate APIx values using the current methodology version.
- Update dashboard caches and expose approved series through authenticated endpoints.

A.3 Web Data Events / Processing Status

Status	Meaning
<i>queued</i>	<i>Job created and waiting for a worker.</i>
<i>running</i>	<i>Source requests are being processed.</i>
<i>partial</i>	<i>Some configured sources or routes succeeded while others failed.</i>
<i>completed</i>	<i>All required tasks completed and results were validated.</i>
<i>rejected</i>	<i>A record or result failed a validation rule.</i>
<i>failed</i>	<i>The job could not complete and requires retry or investigation.</i>

A.4 Final Implementation Note

The project is intentionally designed so that the collection layer, statistical methodology and dashboard can evolve independently. Any future change to source coverage, weights, anomaly rules or index formulas should create a new methodology version and preserve the previous version for comparison. This approach supports reproducibility and protects the historical series from silent changes.

End of Report